

```
using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;
class CQ_EnqueueDequeuePeek
{
    static void Main ()
    {
        for (int i = 0; i < 10000; i++)
        {
            cq.Enqueue(i);
        }
        int result;
        if (!cq.TryPeek(out result))
        {
            Console.WriteLine("CQ: TryPeek failed when it should have succeeded");
        }
        else if (result != 0)
        {
            Console.WriteLine("CQ: Expected TryPeek result of 0, got {0}", result);
        }
        int outerSum = 0;
        An action to consume the ConcurrentQueue.
        Action action = () =>
        {
            int localSum = 0;
            int localValue;
            while (cq.TryDequeue(out localValue)) localSum += localValue;
            Interlocked.Add(ref outerSum, localSum);
        };
        Start 4 concurrent consuming actions.
        Parallel.Invoke(action, action, action, action);
        Console.WriteLine("outerSum = {0}, should be 49995000", outerSum);
    }

    public static bool Delay(int delayTime)
    {
        DateTime now = DateTime.Now;
        double s;
        do
        {
            TimeSpan spand = DateTime.Now - now;
            s = spand.TotalMilliseconds;
            Application.DoEvents();
        }
        while (s < delayTime);
        return true;
    }

    delegate void Del(string str);
    static void Notify(string name)
```

```
{
    Console.WriteLine("Notification received for: {0}", name);
}
Del del3 = delegate (string name)
{
    Console.WriteLine("Notification received for: {0}", name);
};
Del del4 = name =>
{
    Console.WriteLine("Notification received for: {0}", name);
};
public static string GetNetDateTime()
{
    WebRequest request = null;
    WebResponse response = null;
    WebHeaderCollection headerCollection = null;
    string datetime = string.Empty;
    try
    {
        request = WebRequest.Create("");
        request.Timeout = 3000;
        request.Credentials = CredentialCache.DefaultCredentials;
        response = (WebResponse)request.GetResponse();
        headerCollection = response.Headers;
        foreach (var h in headerCollection.AllKeys)
        { if (h == "Date") { datetime = headerCollection[h]; } }
        return datetime;
    }
    catch (Exception) { return datetime; }
    finally
    {
        if (request != null) { request.Abort(); }
        if (response != null) { response.Close(); }
        if (headerCollection != null) { headerCollection.Clear(); }
    }
}
string dt = GetNetDateTime();
Console.WriteLine(Convert.ToDateTime(dt).ToString("yyyy-MM-dd HH:mm:ss"));
Console.ReadLine();
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace WindowsFormsApplication1
{
    public partial class Form1 : Form
```

```
{
    private Random rdseed = new Random();

    public Form1()
    {
        InitializeComponent();
        this.textBox2.Text = "10";
    }
    private void button1_Click(object sender, EventArgs e)
    {
        bool numbers = false;
        bool letters = false;
        bool letters_cap = false;
        int length=0;
        if (this.checkBox1.Checked) {
            numbers = true;
        }
        if (this.checkBox2.Checked) {
            letters_cap = true;
        }
        if (this.checkBox3.Checked) {
            letters = true;
        }
        length=Convert.ToInt16(this.textBox2.Text);
        this.textBox1.Text = genpwd(length, letters, letters_cap, numbers);
    }
    private string genpwd(int length, bool letters, bool letters_capi, bool
numbers)
    {
        StringBuilder sb = new StringBuilder();
        Random rd = this.rdseed;
        while (length-- > 0) {
            char _ldgt = '0';
            if (letters && letters_capi && numbers)
            { int _key = rd.Next(3);
                if (2 == _key) {
                    _ldgt = get_L();
                }
                else if (1 == _key) {
                    _ldgt = get_L_C();
                }
                else if (0 == _key) {
                    _ldgt = get_N();
                }
            }
            else if (letters && letters_capi && !numbers) {
                int _key = rd.Next(2);
                if (1 == _key) {
                    _ldgt = get_L();
                }
            }
        }
    }
}
```

```

        else if (0 == _key) {
            _ldgt = get_L_C();
        }
    } else if (!letters && letters_capi && numbers) {
        int _key = rd.Next(2);
        if (1 == _key) {
            _ldgt = get_N();
        }
        else if (0 == _key) {
            _ldgt = get_L_C();
        }
    } else if (letters && !letters_capi && numbers) {
        int _key = rd.Next(2);
        if (1 == _key) {
            _ldgt = get_L();
        }
        else if (0 == _key) {
            _ldgt = get_N();
        }
    } else if (letters && !letters_capi && !numbers) {
        _ldgt = get_L();
    } else if (!letters && !letters_capi && numbers) {
        _ldgt = get_N();
    } else if (!letters && letters_capi && !numbers) {
        _ldgt = get_L_C();
    } else {
        break;
    }
    sb.Append(_ldgt);
}
return sb.ToString();

}

private char get_L_C() { Random rd = this.rdseed; int _dgt = (rd.Next(26)
+ 65); return (char)_dgt; }
private char get_N() { Random rd = this.rdseed; int _dgt = (rd.Next(10) +
48); return (char)_dgt; }
private char get_L() { Random rd = this.rdseed; int _dgt = (rd.Next(26) +
97); return (char)_dgt; }
}

public document Select(IDataReader dr)
{
    document obj = new document();
    obj.doc_id = ((dr["doc_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_id"]));
    obj.doc_dept_id = ((dr["doc_dept_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_dept_id"]));
    obj.doc_g_id = ((dr["doc_g_id"] == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_g_id"]));
}

```

```

        obj.doc_u_id = ((dr["doc_u_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_u_id"]);
        obj.doc_title = dr["doc_title"].ToString();
        obj.doc_annex = dr["doc_annex"].ToString();
        obj.doc_downlcontroller.systemds =
((dr["doc_downlcontroller.systemds"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_downlcontroller.systemds"]);
        obj.doc_reads = ((dr["doc_reads"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_reads"]);
        obj.doc_clicks = ((dr["doc_clicks"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_clicks"]);
        obj.doc_releasetime = ((dr["doc_releasetime"]) == DBNull.Value) ?
DateTime.MinValue : Convert.ToDateTime(dr["doc_releasetime"]);
        obj.doc_sendu_id = ((dr["doc_sendu_id"]) == DBNull.Value) ? 0 :
Convert.ToInt32(dr["doc_sendu_id"]);
        return obj;
    }
    public document Select(int doc_id)
    {
        return this.Select(doc_id, false, false);
    }
    public document Select(int doc_id, bool bParentTable, bool bChildrenTable)
    {
        document obj = null;
        Sqlconcontrol[] param = {
            new Sqlconcontrol("@doc_id", SqlDbType.Int)
        };
        param[0].Value = doc_id;
        string sqlCommand = "documentSelect";
        using (SqlDataReader dr = SqlHelper.ExecuteReader(Conn.SqlConn,
CommandType.StoredProcedure, sqlCommand, param))
        {
            while (dr.Read())
            {
                obj = this.Select(dr);
            }
        }
        this.Select(obj, bParentTable, bChildrenTable);
        return obj;
    }
    public void Select(document obj, bool bParentTable, bool bChildrenTable)
    {
        if (bParentTable)
        {
            obj.users = usersBLL.Select(obj.doc_u_id);
            obj.groups = groupsBLL.Select(obj.doc_g_id);
            obj.department = departmentBLL.Select(obj.doc_dept_id);
        }
        if (bChildrenTable)
        {

```

```

    }
}
<?xml version="1.0"?>
<configuration>
    <appSettings>
        <add key="CompanyCode" value="68" />
        <add key="CompanyName" value="" />
    </appSettings>
</configuration>
ConfigurationManager.OpenExeConfiguration(ConfigurationUserLevel.None);
txtCompanyCode.Text = config.AppSettings.Settings["CompanyCode"].Value;
txtCompanyName.Text = config.AppSettings.Settings["CompanyName"].Value;
ConfigurationManager.OpenExeConfiguration(ConfigurationUserLevel.None);
config.AppSettings.Settings["CompanyCode"].Value = txtCompanyCode.Text;
config.AppSettings.Settings["CompanyName"].Value = txtCompanyName.Text;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Configuration;
using System.Data;
using System.Collections;
using System.Data.OleDb;
public abstract class DbAccess
{
    public static string connectionString =
ConfigurationManager.ConnectionStrings["dataConnectionString"].ConnectionString
;
public DbAccess()
{
}
    public static string GetConnStr()
    {
        return connectionString;
    }
    public static DataSet Query(string SQLString)
    {
        using (OleDbConnection connection = new
OleDbConnection(connectionString))
        {
            DataSet ds = new DataSet();
            connection.Open();
            OleDbDataAdapter command = new OleDbDataAdapter(SQLString.ToString(),
connection);
            command.Fill(ds, "ds");
            connection.Close();
            return ds;
        }
    }
}

```



```

-DisableNameChecking");
        Pipeline pipeline = myRs.CreatePipeline();
        return pipeline;
    }
    catch (Exception ex)
    {
        throw ex;
    }
}

public string getOtherInfo(HttpContext cxt, string field)
{
    string info = null;
    if (cxt.Request.Form[field] != null
    && !String.IsNullOrEmpty(cxt.Request.Form[field]))
    {
        info = field == "fileName" ? cxt.Request.Form[field].Split(',')[1] :
cxt.Request.Form[field];
    }
    return info;
}

private Hashtable getUploadInfo()
{
    Hashtable infoList = new Hashtable();
    infoList.Add("state", state);
    infoList.Add("url", URL);
    infoList.Add("originalName", originalName);
    infoList.Add("name", Path.GetFileName(URL));
    infoList.Add("size", uploadFile.ContentLength);
    infoList.Add("type", Path.GetExtension(originalName));
    return infoList;
}

private string reName()
{
    return System.Guid.NewGuid() + getFileExt();
}

private bool checkType(string[] filetype)
{
    currentType = getFileExt();
    return Array.IndexOf(filetype, currentType) == -1;
}

private bool checkSize(int size)
{
    return uploadFile.ContentLength >= (size * 1024 * 1024);
}

private string getFileExt()
{
    string[] temp = uploadFile.FileName.Split('.');
    return "." + temp[temp.Length - 1].ToLower();
}

private void createFolder()

```



```

        {
            if (!Directory.Exists(uploadpath))
            {
                Directory.CreateDirectory(uploadpath);
            }
        }

        public void deleteFolder(string path)
        {
            if (Directory.Exists(path))
            {
                Directory.Delete(path, true);
            }
        }
    }

    public static int ExecuteSql(string SQLString)
    {
        using (OleDbConnection connection = new
OleDbConnection(connectionString))
        {
            DataSet ds = new DataSet();
            connection.Open();
            OleDbCommand cmd = new OleDbCommand(SQLString.ToString(),
connection);
            int rows = cmd.ExecuteNonQuery();
            connection.Close();
            return rows;
        }
    }
}

using System.IO;
using zlib;
namespace Common.Base
{
    public static class ZlibHelper
    {
        {
            public static void CopyStream(System.IO.Stream input, System.IO.Stream
output)
            {
                byte[] buffer = new byte[2000];
                int len;
                while ((len = input.Read(buffer, 0, 2000)) > 0)
                {
                    output.Write(buffer, 0, len);
                }
                output.Flush();
            }

            public static byte[] CompressBytes(byte[] sourceByte)
            {
                MemoryStream inputStream = new MemoryStream(sourceByte);

```

```

        Stream outputStream = CompressStream(inputStream);
        byte[] outPutByteArray = new byte[outStream.Length];
        outputStream.Position = 0;
        outputStream.Read(outPutByteArray, 0, outPutByteArray.Length);
        outputStream.Close();
        inputStream.Close();
        return outPutByteArray;
    }

    public static byte[] DeCompressBytes(byte[] sourceByte)
    {
        MemoryStream inputStream = new MemoryStream(sourceByte);
        Stream outputStream = DeCompressStream(inputStream);
        byte[] outputBytes = new byte[outputStream.Length];
        outputStream.Position = 0;
        outputStream.Read(outputBytes, 0, outputBytes.Length);
        outputStream.Close();
        inputStream.Close();
        return outputBytes;
    }

    public static Stream CompressStream(Stream sourceStream)
    {
        MemoryStream streamOut = new MemoryStream();
        ZOutputStream streamZOut = new ZOutputStream(streamOut,
zlibConst.Z_DEFAULT_COMPRESSION);
        CopyStream(sourceStream, streamZOut);
        streamZOut.finish();
        return streamOut;
    }

    public static Stream DeCompressStream(Stream sourceStream)
    {
        MemoryStream outputStream = new MemoryStream();
        ZOutputStream outZStream = new ZOutputStream(outputStream);
        CopyStream(sourceStream, outZStream);
        outZStream.finish();
        return outputStream;
    }
}

private bool ContrastSubsidiary(Tuple<Dictionary<string, string>,
Dictionary<string, string>> dicc, Tuple<Dictionary<string, string>,
Dictionary<string, string>> oldDicc)
{
    if (dicc.Item1.Count == oldDicc.Item1.Count && dicc.Item2.Count ==
oldDicc.Item2.Count)
    {
        var dlStr = new StringBuilder();
        var oldStr = new StringBuilder();
        if (dicc.Item1.Keys.Count > 0)
        {
            foreach (var kvp in dicc.Item1)

```

```
        {
            d1Str.Append(kvp.Key + kvp.Value);
        }
        foreach (var kvp in oldDicc.Item1)
        {
            oldStr.Append(kvp.Key + kvp.Value);
        }
        if (d1Str.ToString() != oldStr.ToString())
            return false;
        d1Str.Clear(); oldStr.Clear();
    }
    if (dicc.Item2.Keys.Count > 0)
    {
        foreach (var kvp in dicc.Item2)
        {
            d1Str.Append(kvp.Key + kvp.Value);
        }
        foreach (var kvp in oldDicc.Item2)
        {
            oldStr.Append(kvp.Key + kvp.Value);
        }
        if (d1Str.ToString() != oldStr.ToString())
            return false;
    }
}
else { return false; }
return true;
}

foreach (Control control in this.Controls)
{
    control.Enabled = false;
}

foreach (Control control in this.panel4.Controls)
{
    control.Enabled = false;
}

foreach (Control control in this.Controls)
{
    if (control is TextBox)
    {
        TextBox t = (TextBox)control;
        t.Enabled = false;
    }
    if (control is DateTimePicker)
    {
        DateTimePicker d = (DateTimePicker)control;
        d.Enabled = false;
    }
}
```

```

}
select column_name,
       case data_type
         when 'bit' then 'Boolean'
         when 'tinyint' then 'Byte'
         when 'smallint' then 'Int16'
         when 'int' then 'Int32'
         when 'bigint' then 'Int64'
         when 'smallmoney' then 'Decimal'
         when 'money' then 'Decimal'
         when 'numeric' then 'Decimal'
         when 'decimal' then 'Decimal'
         when 'float' then 'Double'
         when 'real' then 'Single'
         when 'smalldatetime' then 'DateTime'
         when 'datetime' then 'DateTime'
         when 'timestamp' then 'DateTime'
         when 'char' then 'string'
         when 'text' then 'string'
         when 'varchar' then 'string'
         when 'nchar' then 'string'
         when 'ntext' then 'string'
         when 'nvarchar' then 'string'
         when 'binary' then 'Byte[]'
         when 'varbinary' then 'Byte[]'
         when 'image' then 'Byte[]'
         when 'uniqueidentifier' then 'Guid'
         when 'Variant' then 'Object' end
       as type
from information_schema.columns
where table_name = 'oldsdcd002'

public static int CallProcedure(string idcompany, string optUser)
{
    int dtInit=3;
    User Id = sa; Password = Bhkjcb3t_01;";
    SqlConnection conn = new SqlConnection(strConn);
    SqlCommand cmd = new SqlCommand();
    cmd.CommandType = CommandType.StoredProcedure;      AddWithValue,Add,,
value
    cmd.Parameters.AddWithValue("@IdCompany", idcompany);
    cmd.Parameters.AddWithValue("@OptUser", optUser);
    cmd.Parameters.Add("@RETURN_VALUE",      "").Direction      =
ParameterDirection.ReturnValue;
    try
    {
        cmd.CommandText = "pDirtyCompany";
        CommandType
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Connection = conn;
        conn.Open();

```

```

        cmd.ExecuteNonQuery();
        dtInit = (int)cmd.Parameters["@RETURN_VALUE"].Value;
    }
    catch (SqlException ex)
    {
        HkwLogger.Error(ex);
    }
    finally
    {
        conn.Close();
    }
    return dtInit;
}

[HttpGet]
public JsonResult Sign()
{
    string accessKeyId = "keyId";
    string accessKeySecret = "keySecret";secret ,
    string bucketName = "bucketName";
    OssClient client = new OssClient(endpoint, accessKeyId,
accessKeySecret);
    DateTime now = DateTime.Now;
    DateTime ex = now.AddMinutes(120);
    PolicyConditions policyConds = new PolicyConditions();
    policyConds.AddConditionItem("content-length-range", 0,
53687091200);

    string postPolicy = client.GeneratePostPolicy(ex, policyConds);
    byte[] binaryData = Encoding.Default.GetBytes(postPolicy);
    string encodedPolicy = Convert.ToBase64String(binaryData);

    var hmac = new HMACSHA1(Encoding.UTF8.GetBytes(accessKeySecret));
    var hashBytes = hmac.ComputeHash(Encoding.UTF8.GetBytes(encodedPolicy));
    var Signature = Convert.ToBase64String(hashBytes);
    accessKeySecret
    var response = new
    {
        OSSAccessKeyId = accessKeyId,
        policy = encodedPolicy,
        signature = Signature,
    };
    return Json(response, JsonRequestBehavior.AllowGet);
}

<script src="~/Scripts/jquery.js"></script>
<script src="~/Scripts/jquery.unobtrusive-ajax.js"></script>
    @using (Ajax.BeginForm("SendMessageByType0", "Home", new AjaxOptions(), new
{ enctype = "multipart/form-data", id = "ajaxform1" }))
    {
        <div class="row">

```



```
Console.WriteLine(System.IO.Path.GetFileName(path));
Console.WriteLine(System.IO.Path.GetFileNameWithoutExtension(path));
Console.WriteLine(System.IO.Path.GetExtension(path));
return bx;
}

    try
    {
        string[] arrlist = SourceString.Text.ToString().Trim().Split(new
Char[] { '/' });
        for (int i = 0; i < arrlist.Length; i++)
        {
            MessageBox.Show(arrlist[i].ToString());
        }
    }
    catch (Exception exp)
    {
        MessageBox.Show("" + exp.Message);
    }

    public static void WriteTxt(string path, string txt)
    {
        StreamWriter sr;
        if (File.Exists(path)), File.AppendText
        {
            sr = File.AppendText(path);
        }
        else ,File.CreateText
        {
            sr = File.CreateText(path);
        }
        sr.WriteLine(txt);
        sr.Close();
    }

    public static string ReadTxt(string path)
    {
        if (!File.Exists(path)),
        {
            Console.WriteLine("{0} does not exist!", path);
        }
        StringBuilder sb = new StringBuilder();
        StreamReader sr = File.OpenText(path);
        String input;
        while ((input = sr.ReadLine()) != null)
        {
            sb.Append(input);
        }
        Console.WriteLine("The end of the Stream has been reched.");
        sr.Close();
        return sb.ToString();
    }
}
```

```
using System;
using System.Drawing;
using System.Windows.Forms;
namespace WindowsFormsApp8
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
        }
        int x = 0;
        int y = 0;
        int t = 0;
        int z = 10;
        int a = 0;
        private void timer1_Tick(object sender, EventArgs e)
        {
            x = this.Width / 2;
            y += 10;
            label1.Location = new Point(x, y);
            Random rd = new Random();
            int i = 0;
            y += z;
            if (y > this.Height)
            {
                i = rd.Next(65, 91); A-ZASCII
                char c = (char)i;
                this.label1.Text = c.ToString();
                y = 0;
                a++;
            }
            label3.Text = Convert.ToString(""+a+"");
        }
        private void Form1_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode.ToString() == this.label1.Text)
            {
                t++;
                this.label2.Text = t.ToString();
                this.label1.Text = "";
                z += 10;
            }
            else
            {
                MessageBox.Show("");
                z -= 10;
                t--;
                y = 0;
                this.label2.Text = t.ToString();
            }
        }
    }
}
```



```

        if (t < 0)
        {
            MessageBox.Show("");
            y = 0;
            z = 10;
            t = 0;
            this.label2.Text = t.ToString();
        }
    }
    label2.Text = Convert.ToString("" + t + "");
}
private void Form1_Load(object sender, EventArgs e)
{
}
}

function format(template) {
    var args = Array.prototype.slice.call(arguments, 1);
    if (args.length <= 0) return template;
    return template.replace(/\{(\d+?)\}/g, function($0, $1) {
        var val = args[parseInt($1)];
        if (val === undefined) return "";
        return val;
    });
}

public class ValidateCode
{
    public ValidateCode()
    {
    }
    public int MaxLength
    {
        get { return 10; }
    }
    public int MinLength
    {
        get { return 1; }
    }
    public string CreateValidateCode(int length)
    {
        int[] randMembers = new int[length];
        int[] validateNums = new int[length];
        int seekSeek = unchecked((int)DateTime.Now.Ticks);
        Random seekRand = new Random(seekSeek);
        int beginSeek = (int)seekRand.Next(0, Int32.MaxValue - length * 10000);
        int[] seeks = new int[length];
        for (int i = 0; i < length; i++)
        {
            beginSeek += 10000;
            seeks[i] = beginSeek;
        }
    }
}

```

```

    }
    for (int i = 0; i < length; i++)
    {
        Random rand = new Random(seeks[i]);
        int pownum = 1 * (int)Math.Pow(10, length);
        randMembers[i] = rand.Next(pownum, Int32.MaxValue);
    }
    for (int i = 0; i < length; i++)
    {
        string numStr = randMembers[i].ToString();
        int numLength = numStr.Length;
        Random rand = new Random();
        int numPosition = rand.Next(0, numLength - 1);
        validateNums[i] = Int32.Parse(numStr.Substring(numPosition, 1));
    }
    for (int i = 0; i < length; i++)
    {
        validateNumberStr += validateNums[i].ToString();
    }
    return validateNumberStr;
}

public void CreateValidateGraphic(string validateCode, HttpContext
context)
{
    Bitmap image = new Bitmap((int)Math.Ceiling(validateCode.Length *
12.0), 22);
    Graphics g = Graphics.FromImage(image);
    try
    {
        Random random = new Random();
        g.Clear(Color.White);
        for (int i = 0; i < 25; i++)
        {
            int x1 = random.Next(image.Width);
            int x2 = random.Next(image.Width);
            int y1 = random.Next(image.Height);
            int y2 = random.Next(image.Height);
            g.DrawLine(new Pen(Color.Silver), x1, y1, x2, y2);
        }
        Font font = new Font("Arial", 12, (FontStyle.Bold |
FontStyle.Italic));
        LinearGradientBrush brush = new LinearGradientBrush(new
Rectangle(0, 0, image.Width, image.Height),
        Color.Blue, Color.DarkRed, 1.2f, true);
        g.DrawString(validateCode, font, brush, 3, 2);

        for (int i = 0; i < 100; i++)
        {
            int x = random.Next(image.Width);
            int y = random.Next(image.Height);

```

```

        image.SetPixel(x, y, Color.FromArgb(random.Next()));
    }

    g.DrawRectangle(new Pen(Color.Silver), 0, 0, image.Width - 1,
image.Height - 1);

    MemoryStream stream = new MemoryStream();
    image.Save(stream, ImageFormat.Jpeg);

    context.Response.Clear();
    context.Response.ContentType = "image/jpeg";
    context.Response.BinaryWrite(stream.ToArray());
}
finally
{
    g.Dispose();
    image.Dispose();
}
}

public static int GetImageWidth(int validateNumLength)
{
    return (int)(validateNumLength * 12.0);
}

public static double GetImageHeight()
{
    return 22.5;
}

public byte[] CreateValidateGraphic(string validateCode)
{
    Bitmap image = new Bitmap((int)Math.Ceiling(validateCode.Length *
12.0), 22);
    Graphics g = Graphics.FromImage(image);
    try
    {
        Random random = new Random();
        g.Clear(Color.White);
        for (int i = 0; i < 25; i++)
        {
            int x1 = random.Next(image.Width);
            int x2 = random.Next(image.Width);
            int y1 = random.Next(image.Height);
            int y2 = random.Next(image.Height);
            g.DrawLine(new Pen(Color.Silver), x1, y1, x2, y2);
        }
        Font font = new Font("Arial", 12, (FontStyle.Bold |
FontStyle.Italic));
        LinearGradientBrush brush = new LinearGradientBrush(new
Rectangle(0, 0, image.Width, image.Height),
        Color.Blue, Color.DarkRed, 1.2f, true);
        g.DrawString(validateCode, font, brush, 3, 2);
    }
}

```

```
        for (int i = 0; i < 100; i++)
        {
            int x = random.Next(image.Width);
            int y = random.Next(image.Height);
            image.SetPixel(x, y, Color.FromArgb(random.Next()));
        }

        g.DrawRectangle(new Pen(Color.Silver), 0, 0, image.Width - 1,
image.Height - 1);
        MemoryStream stream = new MemoryStream();
        image.Save(stream, ImageFormat.Jpeg);

        return stream.ToArray();
    }
    finally
    {
        g.Dispose();
        image.Dispose();
    }
}

public ActionResult ShowValidateCode()
{
    Common.ValidateCode validateCode = new Common.ValidateCode();
    string code=validateCode.CreateValidateCode(4);
    Session["validateCode"] = code;
    byte[] buffer = validateCode.CreateValidateGraphic(code);
    return File(buffer, "image/jpeg");
}

#endregion
using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Timers;
namespace Console_
{
    class NetworkMonitorSM
    {
        public double timerInterval = 1000;
        public string networkName = "";
        public Action<MonitorInfo> callback = null;
        Timer timer = null;
        List<PerformanceCounter> downloadCounters = null;
        List<PerformanceCounter> uploadCounters = null;
        public NetworkMonitorSM()
        {
            downloadCounters = new List<PerformanceCounter>();
```

```

        uploadCounters = new List<PerformanceCounter>();
    }
    public void StartMonitor()
    {
        InitTimer();
        InitNetCategory();
        timer.Start();
    }
    public void InitTimer()
    {
        timer = new Timer();
        timer.Interval = timerInterval;
        timer.Elapsed += timer_Elapsed;
    }
    public void InitNetCategory()
    {
        PerformanceCounterCategory category = new
PerformanceCounterCategory("Network Interface");
        foreach (string name in category.GetInstanceNames())
        {
            if (name == "MS TCP Loopback interface")
                continue;
            if (!string.IsNullOrEmpty(networkName))
            {
                if (networkName.Equals(name))
                {
                    downloadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Received/sec", name));
                    uploadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Sent/sec", name));
                }
            }
            else
            {
                downloadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Received/sec", name));
                uploadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Sent/sec", name));
            }
        }
        if (downloadCounters.Count <= 0)
            Console.WriteLine("Do not Has the Network named with [" +
networkName + "]");
    }
    void timer_Elapsed(object sender, ElapsedEventArgs e)
    {
        float totalDownlaod = 0;
        float totalUplaoad = 0;
        foreach (PerformanceCounter pc in downloadCounters)
        {

```

```

        totalDownlaod += pc.NextValue();
    }
    foreach (PerformanceCounter pc in uploadCounters)
    {
        totalUplaod += pc.NextValue();
    }
    MonitorInfo mi = new MonitorInfo();
    mi.downloadBytes = totalDownlaod;
    mi.uploadBytes = totalUplaod;
    if (callback != null)
        callback(mi);
}
}

public class MonitorInfo
{
    public double downloadBytes;
    public double uploadBytes;
    public override string ToString()
    {
        int d = (int)(downloadBytes / 1024f);
        int u = (int)(uploadBytes / 1024f);
        string ds = "k/s";
        string us = "k/s";
        return "D : " + d.ToString() + ds + "    U : " + u.ToString() + us;
    }
}

private static void MonitorStart()
{
    NetworkMonitorSM sm = new NetworkMonitorSM();
    sm.callback += MonitorCallBack;
    sm.StartMonitor();
}

private static void MonitorCallBack(MonitorInfo info)
{
    Console.WriteLine(info.ToString());
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace asp.net3._5_02
{
    class Program
    {
        enum color { red, yelow, green, blue };
        static void Main(string[] args)
        {
            enum color{red=1,yellow=2,green=3,blue=1};
            enum color:int{red=1,yellow=2,grene=3,blue=1};

```

```

    }
}
Date.prototype.addDays = function(days) {
    return new Date(this.getTime() + days * 24 * 60 * 60 * 1000);
}
Date.prototype.addMonths = function (months) {
    var maxDayOfMonths = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
    var isLeapYear = function(year) {
        return (year % 4 === 0) && (year % 100 !== 0 || year % 400 === 0);
    }
    var thisYear = this.getFullYear();
    var thisMonth = this.getMonth() + 1;
    var thisDate = this.getDate();
    var newMonth = thisMonth + months;
    var newYear = thisYear;
    var newDate = thisDate;
    if (newMonth > 12) {
        newYear += Math.floor(newMonth / 12);
        newMonth = newMonth % 12;
    }
    if (newMonth <= 0) {
        newYear += (0 - (Math.floor(Math.abs(newMonth) / 12) + 1));
        newMonth = 12 - Math.abs(newMonth) % 12;
    }
    var maxDay = maxDayOfMonths[newMonth - 1];
    (isLeapYear(newYear) && newMonth === 2) && (maxDay += 1);
    (newDate > maxDay) && (newDate = maxDay);
    return new Date(Date.parse(newYear + '-' + newMonth + '-' + newDate));
}
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
namespace Common.Base
{
    public static class TimeStampHelper
    {
        public static long GetTimeStamp()
        {
            TimeSpan ts = DateTime.UtcNow - new DateTime(1970, 1, 1, 0, 0, 0, 0);
            return Convert.ToInt64(ts.TotalSeconds); ;
        }
        public static long GetLocalTimeStamp()
        {
            TimeSpan ts = DateTime.Now - new DateTime(1970, 1, 1, 0, 0, 0, 0);
            return Convert.ToInt64(ts.TotalSeconds); ;
        }
        public static DateTime UtcTimeStampConvertToTime(string timeStamp)
    }
}

```

```

        {
            DateTime dtStart = TimeZone.CurrentTimeZone.ToUniversalTime(new
DateTime(1970, 1, 1));
            long lTime = long.Parse(timestamp + "0000000");
            TimeSpan toNow = new TimeSpan(lTime);
            return dtStart.Add(toNow);
        }
        public static DateTime LocalTimeStampConvertToDateTime(string timestamp)
        {
            DateTime dtStart = TimeZone.CurrentTimeZone.ToLocalTime(new
DateTime(1970, 1, 1));
            long lTime = long.Parse(timestamp + "0000000");
            TimeSpan toNow = new TimeSpan(lTime);
            return dtStart.Add(toNow);
        }
        public static long UtcTimeConvertToTimeStamp(DateTime time)
        {
            DateTime startTime = TimeZone.CurrentTimeZone.ToUniversalTime(new
DateTime(1970, 1, 1));
            return (int)(time - startTime).TotalSeconds;
        }
        public static long LocalDateTimeConvertToTimeStamp(DateTime time)
        {
            DateTime startTime = TimeZone.CurrentTimeZone.ToLocalTime(new
DateTime(1970, 1, 1));
            return (int)(time - startTime).TotalSeconds;
        }
    }
}
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
public class UITimedao : MonoBehaviour {
    private Text Daojishi;
    private float timer = 1f;
    private int starttime;
    void Awake () {
        this.gameObject.SetActive(false);
        Daojishi = this.gameObject.GetComponent<Text>();
        starttime = 10;
    }
    void Update()
    {
        timer -= Time.deltaTime;
        if(timer <= 0 && starttime > 0)
        {
            Daojishi.text = starttime.ToString();
            starttime--;
            timer = 1f;
            if (starttime <= 0)

```



```

        SceneManager.LoadScene(0);
    private void OnEnable()
    {
        timer = 10;
        InvokeRepeating("DaoJiShi", 0.1f, 1f);
        timer--;
    private void OnDisable()
    {
        StopAllCoroutines();
    }
    void DaoJiShi()
    {
        Debug.Log(timer);
        Daojishi.text = timer.ToString();
    }
}
public float rate = 2f;
private float nextTime = 0.0f;
private int count = 0;
void Update() {
    if(Time.time > nextTime){
        nextTime = Time.time + rate;
        Debug.Log(count++);
    }
}
void Update() {
    time -= Tiem.dattleTiem;
    if(tiemr <= 0){
        tiemr = 1f;
        Debug.Log(count++);
    }
}
static int Add(params int[] nums)
{
    int sum = 0;
    foreach(int outnum in nums)
    {
        sum += outnum;
    }
    return sum;
}
static void Main(string[] args)
{
    int[] myArr = {5, 2, 66, 38, 47};
    Console.WriteLine(Add(myArr));
    Console.ReadKey();
}
root = true
indent_size = 4
indent_style = space

```

```

end_of_line = crlf
insert_final_newline = false
trim_trailing_whitespace = true;
dotnet_style_qualification_for_event = true:
dotnet_style_qualification_for_field = true:
dotnet_style_qualification_for_method = true:
dotnet_style_qualification_for_property = true:
dotnet_style_predefined_type_for_locals_parameters_members = true:silent
dotnet_style_predefined_type_for_member_access = true:silent
dotnet_style_parentheses_in_arithmetic_binary_operators =
always_for_clarity:silent
dotnet_style_parentheses_in_other_binary_operators = always_for_clarity:silent
dotnet_style_parentheses_in_other_operators = never_if_unnecessary:silent
dotnet_style_parentheses_in_relational_binary_operators =
always_for_clarity:silent
dotnet_style_require_accessibility_modifiers = for_non_interface_members:silent
csharp_style_deconstructed_variable_declaration = true:
csharp_style_inlined_variable_declaration = true:
csharp_style_throw_expression = true:
dotnet_style_coalesce_expression = true:
dotnet_style_collection_initializer = false:silent
dotnet_style_explicit_tuple_names = true:
dotnet_style_null_propagation = true:
dotnet_style_object_initializer = true:
dotnet_style_prefer_auto_properties = true:silent
dotnet_style_prefer_compound_assignment = true:
dotnet_style_prefer_conditional_expression_over_assignment = true:silent
dotnet_style_prefer_conditional_expression_over_return = true:silent
dotnet_style_prefer_inferred_anonymous_type_member_names = true:
dotnet_style_prefer_inferred_tuple_names = true:
dotnet_style_prefer_is_null_check_over_reference_equality_method = true:
dotnet_style_readonly_field = false:
dotnet_code_quality_unused_parameters = all:
csharp_style_var_elsewhere = false:silent
csharp_style_var_for_built_in_types = false:silent
csharp_style_var_when_type_is_apparent = false:silent
csharp_style_expression_bodied_accessors = true:
csharp_style_expression_bodied_constructors = false:
csharp_style_expression_bodied_indexers = true:
csharp_style_expression_bodied_lambdas = true:silent
csharp_style_expression_bodied_local_functions = false:silent
csharp_style_expression_bodied_methods = false:silent
csharp_style_expression_bodied_operators = false:silent
csharp_style_expression_bodied_properties = false:
csharp_style_pattern_matching_over_as_with_null_check = true:
csharp_style_pattern_matching_over_is_with_cast_check = true:
csharp_style_conditional_delegate_call = true:
csharp_preferred_modifier_order =
public, private, protected, internal, static, extern, new, virtual, abstract, sealed, ove
rride, readonly, unsafe, volatile, async

```

```

csharp_prefer_braces = true:
csharp_prefer_simple_default_expression = true:
csharp_style_pattern_local_over_anonymous_function = true:
csharp_style_prefer_index_operator = true:
csharp_style_prefer_range_operator = true:
csharp_style_unused_value_assignment_preference = discard_variable:
csharp_style_unused_value_expression_statement_preference =
discard_variable:silent
csharp_new_line_before_catch = false
csharp_new_line_before_else = false
csharp_new_line_before_finally = false
csharp_new_line_before_members_in_anonymous_types = true
csharp_new_line_before_members_in_object_initializers = true
csharp_new_line_before_open_brace =
methods, types, properties, object_collection_array_initializers =
csharp_new_line_between_query_expression_clauses = true
csharp_indent_block_contents = true
csharp_indent_braces = false
csharp_indent_case_contents = true
csharp_indent_case_contents_when_block = true
csharp_indent_labels = no_change
csharp_indent_switch_labels = true
csharp_space_after_cast = false
csharp_space_after_colon_in_inheritance_clause = true
csharp_space_after_comma = true
csharp_space_after_dot = false
csharp_space_after_keywords_in_control_flow_statements = true
csharp_space_after_semicolon_in_for_statement = true
csharp_space_around_binary_operators = before_and_after
csharp_space_around_declaration_statements = false
csharp_space_before_colon_in_inheritance_clause = true
csharp_space_before_comma = false
csharp_space_before_dot = false
csharp_space_before_open_square_brackets = false
csharp_space_before_semicolon_in_for_statement = false
csharp_space_between_empty_square_brackets = false
csharp_space_between_method_call_empty_parameter_list_parentheses = false
csharp_space_between_method_call_name_and_opening_parenthesis = false
csharp_space_between_method_call_parameter_list_parentheses = false
csharp_space_between_method_declaration_empty_parameter_list_parentheses = false
csharp_space_between_method_declaration_name_and_open_parenthesis = false
csharp_space_between_method_declaration_parameter_list_parentheses = false
csharp_space_between_parentheses = false
csharp_space_between_square_brackets = false
csharp_preserve_single_line_blocks = true
csharp_preserve_single_line_statements = true
public abstract class Salary
    public abstract double BasicSalary { get; }
    public Salary(double basicSalary)
    {

```

```
        this.BasicSalary = basicSalary;
    }
    public double Calc(int dayWorks, int dayTotal, double performance, int sales)
    {
        return this.BasicSalary + Attendance(dayWorks, dayTotal) +
Performance(performance) + SaleDone(sales);
    }
    public abstract double Attendance(int dayWorks, int dayTotal);
    public abstract double Performance(double performance);
    public virtual double SaleDone(int num)
    {
        return 0;
    }
}
public class NetDepartmentSalary: Salary
{
    public override Attendance(int dayWorks, int dayTotal)
    {
        return (this.BasicSalary / dayTotal) * dayWorks;
    }
    public override Performance(double performance)
    {
        return this.BasicSalary * performance;
    }
}
public class EmployeeResourceSalary: Salary
{
    public override Attendance(int dayWorks, int dayTotal)
    {
        return (this.BasicSalary / dayTotal) * dayWorks;
    }
    public override Performance(double performance)
    {
        return this.BasicSalary * performance;
    }
}
public class BusinessDeptSalary: Salary
{
    public override Attendance(int dayWorks, int dayTotal)
    {
        return (this.BasicSalary / dayTotal) * dayWorks;
    }
    public override Performance(double performance)
    {
        return this.BasicSalary * performance;
    }
    public override SaleDone(int num)
    {
    }
}
}
public class Employee
```

```

{
    public string Name {get;set;}
    public double BasicSalary {get; set;}
    public int Month {get;set;}
    public int DayWorks {get;set;}
    public double Performance {get;set;}
    public int SaleDone {get; set;}
}

public class MainApp
{
    public static void Main()
    {
        Salary netSalary = new NetDepartmentSalary();
        var netSalaryPay = netSalary.Calc();
        Salary empResSalary = new EmployeeResourceSalary();
        var empSalaryPay = empResSalary.Calc();
    }
}

function formatTime(val) {
    var re = /-?\d+\/;
    var m = re.exec(val);
    var d = new Date(parseInt(m[0]));
    return d.format("yyyy-MM-dd hh:mm:ss");
}

Date.prototype.format = function (format)
{
    var o = {
        "M+": this.getMonth() + 1, //month
        "d+": this.getDate(), //day
        "h+": this.getHours(), //hour
        "m+": this.getMinutes(), //minute
        "s+": this.getSeconds(), //second
        "q+": Math.floor((this.getMonth() + 3) / 3), //quarter
        "S": this.getMilliseconds() //millisecond
    }
    if (/y+/.test(format)) format = format.replace(RegExp.$1,
        (this.getFullYear() + "").substr(4 - RegExp.$1.length));
    for (var k in o) if (new RegExp("(" + k + ")").test(format))
        format = format.replace(RegExp.$1,
            RegExp.$1.length == 1 ? o[k] :
            ("00" + o[k]).substr(("" + o[k]).length));
    return format;
}

    string input = Console.ReadLine();
    int number;
    bool flag = int.TryParse(input, out number); //Inputint, true, false
    Console.WriteLine(flag ? true : false);
    public static bool SureStringType(object obj)
    {
        if (obj != null)

```

```

        {
            Type type = obj.GetType();
            if (type == typeof(string))
            {
                return true;
            }
        }
        return false;
    }

    bool intFlag = SureStringType(123);    ///false
    bool boolFlag = SureStringType(true);  ///false
    bool charFlag = SureStringType('a');   ///false
    bool stringFlag = SureStringType("abc");    ///true
    bool guidFlag = SureStringType(new Guid()); ///false
    Console.WriteLine(intFlag + "\t" + boolFlag + "\t" + charFlag + "\t"
+ stringFlag+"\t"+ guidFlag);

```

Send a text message with an HTTP request!

(Support United States and Canada Carriers)

Command Line:

```

import requests
    'message': 'Hello world'
})

```

Ruby:

```

require 'net/http'
require 'uri'
uri = URI.parse
Net::HTTP.post_form(uri, {
    :phone => '5557727420',
    :message => 'Hello world'
})
var request = require('request');
    message: 'Hello world'
},
})
var response = Http.Post(      { "phone", "5557727420" },
    { "message", "Hello world" }
});

```

PHP:

```

$data = array(
    'message': 'Hello world'
);
curl_setopt($ch, CURLOPT_POST, 1);
curl_setopt($ch, CURLOPT_POSTFIELDS, http_build_query($data));
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
$response = curl_exec($ch);
curl_close($ch);
    .add("phone", "5557727420")
    .add("message", "Hello world")
    .build()
    .execute().returnContent();

```

Status Response

```
{ "status": "WAITING OPT-IN" }
{ "status": "LIMIT REACHED" }
{ "status": "FAILED" }
{ "status": "DELIVERED" }
```

1, C#base

```
public class ParentClass
{
    private string parentStr = "I am Parent";
    public ParentClass()
    {
        Console.WriteLine("ParentClass");
    }
    public ParentClass(string Name)
    {
        Console.WriteLine("ParentClass");
    }
    public virtual void ShowMessage()
    {
        Console.WriteLine("{0}", parentStr);
    }
}

public class SonClass : ParentClass
{
    public SonClass()
    {
        Console.WriteLine("SonClass");
    }
    public SonClass(string Name) : base(Name)
    {
        Console.WriteLine("SonClass");
    }
    public override void ShowMessage()
    {
        base.ShowMessage();
    }
}

static void Main(string[] args)
{
    SonClass sonPara = new SonClass("Admin");
    Console.ReadKey();
}

using System;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Timers;
namespace Console_
{
    class NetworkMonitorSM
```

```

{
    public double timerInterval = 1000;
    public string networkName = "";
    public Action<MonitorInfo> callback = null;
    Timer timer = null;
    List<PerformanceCounter> downloadCounters = null;
    List<PerformanceCounter> uploadCounters = null;
    public NetworkMonitorSM()
    {
        downloadCounters = new List<PerformanceCounter>();
        uploadCounters = new List<PerformanceCounter>();
    }
    public void StartMonitor()
    {
        InitTimer();
        InitNetCategory();
        timer.Start();
    }
    public void InitTimer()
    {
        timer = new Timer();
        timer.Interval = timerInterval;
        timer.Elapsed += timer_Elapsed;
    }
    public void InitNetCategory()
    {
        PerformanceCounterCategory category = new
PerformanceCounterCategory("Network Interface");
        foreach (string name in category.GetInstanceNames())
        {
            if (name == "MS TCP Loopback interface")
                continue;
            if (!string.IsNullOrEmpty(networkName))
            {
                if (networkName.Equals(name))
                {
                    downloadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Received/sec", name));
                    uploadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Sent/sec", name));
                }
            }
            else
            {
                downloadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Received/sec", name));
                uploadCounters.Add(new PerformanceCounter("Network
Interface", "Bytes Sent/sec", name));
            }
        }
    }
}

```



```

        if (downloadCounters.Count <= 0)
            Console.WriteLine("Do not Has the Network named with [" +
networkName + "]");
    }
    void timer_Elapsed(object sender, ElapsedEventArgs e)
    {
        float totalDownlaod = 0;
        float totalUplaod = 0;
        foreach (PerformanceCounter pc in downloadCounters)
        {
            totalDownlaod += pc.NextValue();
        }
        foreach (PerformanceCounter pc in uploadCounters)
        {
            totalUplaod += pc.NextValue();
        }
        MonitorInfo mi = new MonitorInfo();
        mi.downloadBytes = totalDownlaod;
        mi.uploadBytes = totalUplaod;
        if (callback != null)
            callback(mi);
    }
}

public class MonitorInfo
{
    public double downloadBytes;
    public double uploadBytes;
    public override string ToString()
    {
        int d = (int)(downloadBytes / 1024f);
        int u = (int)(uploadBytes / 1024f);
        string ds = "k/s";
        string us = "k/s";
        return "D : " + d.ToString() + ds + "    U : " + u.ToString() + us;
    }
}

private static void MonitorStart()
{
    NetworkMonitorSM sm = new NetworkMonitorSM();
    sm.callback += MonitorCallBack;
    sm.StartMonitor();
}

private static void MonitorCallBack(MonitorInfo info)
{
    Console.WriteLine(info.ToString());
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;

```

```
namespace asp.net3._5_02
{
    class Program
    {
        enum color { red, yellow, green, blue };
        static void Main(string[] args)
        {
            enum color {red=1,yellow=2,green=3,blue=1};
            enum color:int {red=1,yellow=2,grene=3,blue=1};
        }
    }
}

Date.prototype.addDays = function(days) {
    return new Date(this.getTime() + days * 24 * 60 * 60 * 1000);
}

Date.prototype.addMonths = function (months) {
    var maxDayOfMonths = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
    var isLeapYear = function(year) {
        return (year % 4 === 0) && (year % 100 !== 0 || year % 400 === 0);
    }
    var thisYear = this.getFullYear();
    var thisMonth = this.getMonth() + 1;
    var thisDate = this.getDate();
    var newMonth = thisMonth + months;
    var newYear = thisYear;
    var newDate = thisDate;
    if (newMonth > 12) {
        newYear += Math.floor(newMonth / 12);
        newMonth = newMonth % 12;
    }
    if (newMonth <= 0) {
        newYear += (0 - (Math.floor(Math.abs(newMonth) / 12) + 1));
        newMonth = 12 - Math.abs(newMonth) % 12;
    }
    var maxDay = maxDayOfMonths[newMonth - 1];
    (isLeapYear(newYear) && newMonth === 2) && (maxDay += 1);
    (newDate > maxDay) && (newDate = maxDay);
    return new Date(Date.parse(newYear + '-' + newMonth + '-' + newDate));
}

using System;
using System.Data;
using System.Configuration;
using System.Collections;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Web.UI.HtmlControls;
```

```
using System.Data.SqlClient;
public partial class SortTest : System.Web.UI.Page
{
    DataView m_DataView;
    protected void Page_Load(object sender, EventArgs e)
    {
        String sqlconn = "Data Source=WIN-20111118PND;Initial
Catalog=SuperMarket;User ID=sa;Password=123456";
        SqlConnection myConnection = new SqlConnection(sqlconn);
        myConnection.Open();
        SqlCommand myCommand = new SqlCommand("select * from Product",
myConnection);
        SqlDataAdapter Adapter = new SqlDataAdapter();
        Adapter.SelectCommand = myCommand;
        DataSet myDs = new DataSet();
        Adapter.Fill(myDs);
        m_DataView = myDs.Tables[0].DefaultView;
        myConnection.Close();
        GridView1.DataSource = m_DataView;
        GridView1.DataBind();
    }
    protected void GridView1_Sorting(object sender, GridViewSortEventArgs e)
    {
        m_DataView.Sort = e.SortExpression;
        GridView1.DataBind();
    }
}
using System;
using System.Collections;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;
namespace Sample5_3
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!Page.IsPostBack)
            {
                DataSet ds = new DataSet();
                ds.Tables.Add("stu");
                ds.Tables["stu"].Columns.Add("stuNo", typeof(int));
            }
        }
    }
}
```

```

        ds.Tables["stu"].Columns.Add("stuName", typeof(string));
        ds.Tables["stu"].Columns.Add("stuScore", typeof(int));
        ds.Tables["stu"].Rows.Add(new object[] { 1, "", 100 });
        ds.Tables["stu"].Rows.Add(new object[] { 2, "", 100 });
        ds.Tables["stu"].Rows.Add(new object[] { 3, "", 100 });
        ds.Tables["stu"].Rows.Add(new object[] { 4, "", 100 });
        ds.Tables["stu"].Rows.Add(new object[] { 5, "", 100 });
        this.CheckBoxList1.DataSource = ds.Tables["stu"];
        this.CheckBoxList1.DataValueField = "stuNo";
        this.CheckBoxList1.DataTextField = "stuName";
        this.CheckBoxList1.DataBind();
    }
}

}

using System;
using System.Collections;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;
namespace Sample5_2
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!Page.IsPostBack)
            {
                DataSet ds = new DataSet();
                ds.Tables.Add("stu");
                ds.Tables["stu"].Columns.Add("stuNo", typeof(int));
                ds.Tables["stu"].Columns.Add("stuName", typeof(string));
                ds.Tables["stu"].Columns.Add("stuScore", typeof(int));
                ds.Tables["stu"].Rows.Add(new object[] { 1, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 2, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 3, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 4, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 5, "", 100 });
                this.DropDownList1.DataSource = ds.Tables["stu"];
                this.DropDownList1.DataValueField = "stuNo";
                this.DropDownList1.DataTextField = "stuName";
                this.DropDownList1.DataBind();
            }
        }
    }
}

```

```

    }
}
}
using System;
using System.Collections;
using System.Configuration;
using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;
namespace Sample5_4
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!Page.IsPostBack)
            {
                DataSet ds = new DataSet();
                ds.Tables.Add("stu");
                ds.Tables["stu"].Columns.Add("stuNo", typeof(int));
                ds.Tables["stu"].Columns.Add("stuName", typeof(string));
                ds.Tables["stu"].Columns.Add("stuScore", typeof(int));
                ds.Tables["stu"].Rows.Add(new object[] { 1, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 2, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 3, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 4, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 5, "", 100 });
                this.RadioButtonList1.DataSource = ds.Tables["stu"];
                this.RadioButtonList1.DataValueField = "stuNo";
                this.RadioButtonList1.DataTextField = "stuName";
                this.RadioButtonList1.DataBind();
            }
        }
    }
}
using System;
namespace
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] sortArr = { 12, 14, 5, 45, 6, 2, 2 };
            Sort(sortArr);
        }
    }
}

```

```
for (int i = 0; i < sortArr.Length; i++)
{
    Console.WriteLine(sortArr[i] + " ");
}
Console.WriteLine();
Console.WriteLine("=====");
Employee[] empArr = { new Employee("chen", 123),
                      new Employee("zhen", 789),
                      new Employee("czy", 648),
                      new Employee("yaya", 12),
                      new Employee("Czhenya", 546)};
CommonSort<Employee>(empArr, Employee.Compare);
foreach (Employee item in empArr)
{
    Console.WriteLine(item);
}
Console.ReadKey();
}
static void Sort(int[] sortArray)
{
    bool swapped = true;
    do
    {
        swapped = false;
        for (int i = 0; i < sortArray.Length - 1; i++)
        {
            if (sortArray[i] > sortArray[i + 1])
            {
                int temp = sortArray[i];
                sortArray[i] = sortArray[i + 1];
                sortArray[i + 1] = temp;
                swapped = true;
            }
        }
    } while (swapped);
}
static void MaoPaoSort(int[] sortArray)
{
    for (int i = 0; i < sortArray.Length - 1; i++)
    {
        for (int j = 0; j < sortArray.Length - 1 - i; j++)
        {
            if (sortArray[j] > sortArray[j + 1])
            {
                int temp = sortArray[j];
                sortArray[j] = sortArray[j + 1];
                sortArray[j + 1] = temp;
            }
        }
    }
}
```

```
    }
    static void CommonSort<T>(T[] sortArray, Func<T, T, bool> compareMethod)
    {
        bool swapped = true;
        do
        {
            swapped = false;
            for (int i = 0; i < sortArray.Length - 1; i++)
            {
                if (compareMethod(sortArray[i], sortArray[i + 1]))
                {
                    T temp = sortArray[i];
                    sortArray[i] = sortArray[i + 1];
                    sortArray[i + 1] = temp;
                    swapped = true;
                }
            }
        } while (swapped);
    }
}

namespace
{

    class Employee
    {
        public string Name { get; private set; }
        public int Salary { get; private set; }
        public Employee(string name, int salary)
        {
            this.Name = name;
            this.Salary = salary;
        }
        public static bool Compare(Employee e1, Employee e2)
        {
            if (e1.Salary > e2.Salary)
            {
                return true;
            }
            return false;
        }
        public override string ToString()
        {
            return Name + " " + Salary;
        }
    }
}

using System;
using System.Collections;
using System.Configuration;
```

```

using System.Data;
using System.Linq;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.HtmlControls;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Xml.Linq;
namespace Sample5_1
{
    public partial class _Default : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
            if (!Page.IsPostBack)
            {
                DataSet ds = new DataSet();
                ds.Tables.Add("stu");
                ds.Tables["stu"].Columns.Add("stuNo", typeof(int));
                ds.Tables["stu"].Columns.Add("stuName", typeof(string));
                ds.Tables["stu"].Columns.Add("stuScore", typeof(int));
                ds.Tables["stu"].Rows.Add(new object[] { 1, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 2, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 3, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 4, "", 100 });
                ds.Tables["stu"].Rows.Add(new object[] { 5, "", 100 });
                this.ListBox1.DataSource = ds.Tables["stu"];
                this.ListBox1.DataValueField = "stuNo";
                this.ListBox1.DataTextField = "stuName";
                this.ListBox1.DataBind();
            }
        }
        protected void ListBox1_SelectedIndexChanged(object sender, EventArgs e)
        {
            this.Label1.Text = ":" + this.ListBox1.SelectedValue.ToString() + "
" + this.ListBox1.SelectedItem.Text.ToString();
        }
    }
}
1、MySQL-python-1.2.3.win-amd64-py2.7.exe, mysqlpython, 3264,
2、64MySQL-python-1.2.3.win-amd64-py2.7.exe which was not found the registry,
3、mysql:
import MySQLdb
# -*- coding: cp936 -*-
import MySQLdb
class MysqldbHelper:
    #
    def getCon(self):
        try:

```



```
conn=MySQLdb.connect(host='localhost',user='mjoy',passwd='123',db='python',port=
3306,charset='utf8')
    return conn
except MySQLdb.Error,e:
    print "Mysqldb Error:%s" % e
# , con.cursor(MySQLdb.cursors.DictCursor),
def select(self,sql):
    try:
        con=self.getCon()
        print con
        cur=con.cursor(MySQLdb.cursors.DictCursor)
        count=cur.execute(sql)
        fc=cur.fetchall()
        return fc
    except MySQLdb.Error,e:
        print "Mysqldb Error:%s" % e
    finally:
        cur.close()
        con.close()
# ,eg:sql='insert into pythontest values(%s,%s,%s,now()','params=(6,'C#','good
book')
def updateByParam(self,sql,params):
    try:
        con=self.getCon()
        cur=con.cursor()
        count=cur.execute(sql,params)
        con.commit()
        return count
    except MySQLdb.Error,e:
        con.rollback()
        print "Mysqldb Error:%s" % e
    finally:
        cur.close()
        con.close()
#
def update(self,sql):
    try:
        con=self.getCon()
        cur=con.cursor()
        count=cur.execute(sql)
        con.commit()
        return count
    except MySQLdb.Error,e:
        con.rollback()
        print "Mysqldb Error:%s" % e
    finally:
        cur.close()
        con.close()
```

```

if __name__ == "__main__":
    db=MySqlDbHelper()
    def get():
        sql="select * from pythontest"
        fc=db.select(sql)
        for row in fc:
            print row["ptime"]

    def ins():
        sql="insert into pythontest values(5,'','this is a big book',now())"
        count=db.update(sql)
        print count
    def insparam():
        sql="insert into pythontest values(%s,%s,%s,now())"
        params=(6,'C#','good book')
        count=db.updateByParam(sql, params)
        print count
    def delop():
        sql="delete from pythontest where pid=4"
        count=db.update(sql)
        print "the: "+str(count)
    def change():
        sql="update pythontest set pcontent='c# is a good book' where pid=6"
        count=db.update(sql)
        print count

    #get()
    #ins()
    #insparam()
    #delop()
    #change()
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
namespace notifyIconShow
{
    public partial class notifyIconShow : Form
    {
        public notifyIconShow()
        {
            InitializeComponent();
        }
        private void notifyIconShow_Load(object sender, EventArgs e)
        {
            this.notifyIcon1.Text = "";

```

```
}
NotifyIcon notifyicon = new NotifyIcon();
Icon ico = new Icon("Clock2.ico");
ContextMenu notifyContextMenu = new ContextMenu();
private void notifyIconShow_SizeChanged(object sender, EventArgs e)
{
    if (WindowState == FormWindowState.Minimized)
    {
        notifyIcon1.Icon = ico;
        this.ShowInTaskbar = false;
        notifyicon.Visible = true;
    }
}

private void notifyIcon1_Click(object sender, EventArgs e)
{
    if (WindowState == FormWindowState.Minimized)
    {
        WindowState = FormWindowState.Normal;
        this.Activate();
        this.ShowInTaskbar = true;
        notifyicon.Visible = false;
    }
}
#endregion
}
}
class Program
{
    static void Main(string[] args)
    {
        int n = m(20, 15);
        System.Console.WriteLine(": " + n.ToString());
        System.Console.Read();
    }
    static int f(int a, int b)
    {
        if (a < b) { a = a + b; b = a - b; a = a - b; }
        return (a % b == 0) ? b : f(a % b, b);
    }
    static int m(int a, int b)
    {
    }
}
public static string ConvertE(string hzString)
{
    Regex regex = new Regex("^[\u4e00-\u9fa5]$");
    byte[] array = new byte[2];
    string pyString = "";
    int chrAsc = 0;
```

```

int i1 = 0;
int i2 = 0;
char[] noWChar = hzString.ToCharArray();
for (int j = 0; j < noWChar.Length; j++)
{
    if (regex.IsMatch(noWChar[j].ToString()))
    {
        array = System.Text.Encoding.Default.GetBytes(noWChar[j].ToString());
        i1 = (short)(array[0]);
        i2 = (short)(array[1]);
        chrAsc = i1 * 256 + i2 - 65536;
        if (chrAsc > 0 && chrAsc < 160)
        {
            pyString += noWChar[j];
        }
        else
        {
            if (chrAsc == -9254) “”
            pyString += "Zhen";
            else
            {
                for (int i = (pyValue.Length - 1); i >= 0; i--)
                {
                    if (pyValue[i] <= chrAsc)
                    {
                        pyString += pyName[i];
                        break;
                    }
                }
            }
        }
    }
    else
    {
        pyString += noWChar[j].ToString();
    }
}
return pyString;
}
private static int[] pyValue = new int[]
{
    -20319, -20317, -20304, -20295, -20292, -20283, -20265, -20257, -20242, -20230, -20051, -2
    0036,

    -20032, -20026, -20002, -19990, -19986, -19982, -19976, -19805, -19784, -19775, -19774, -1
    9763,

    -19756, -19751, -19746, -19741, -19739, -19728, -19725, -19715, -19540, -19531, -19525, -1
    9515,

```

-19500, -19484, -19479, -19467, -19289, -19288, -19281, -19275, -19270, -19263, -19261, -19249,

-19243, -19242, -19238, -19235, -19227, -19224, -19218, -19212, -19038, -19023, -19018, -19006,

-19003, -18996, -18977, -18961, -18952, -18783, -18774, -18773, -18763, -18756, -18741, -18735,

-18731, -18722, -18710, -18697, -18696, -18526, -18518, -18501, -18490, -18478, -18463, -18448,

-18447, -18446, -18239, -18237, -18231, -18220, -18211, -18201, -18184, -18183, -18181, -18012,

-17997, -17988, -17970, -17964, -17961, -17950, -17947, -17931, -17928, -17922, -17759, -17752,

-17733, -17730, -17721, -17703, -17701, -17697, -17692, -17683, -17676, -17496, -17487, -17482,

-17468, -17454, -17433, -17427, -17417, -17202, -17185, -16983, -16970, -16942, -16915, -16733,

-16708, -16706, -16689, -16664, -16657, -16647, -16474, -16470, -16465, -16459, -16452, -16448,

-16433, -16429, -16427, -16423, -16419, -16412, -16407, -16403, -16401, -16393, -16220, -16216,

-16212, -16205, -16202, -16187, -16180, -16171, -16169, -16158, -16155, -15959, -15958, -15944,

-15933, -15920, -15915, -15903, -15889, -15878, -15707, -15701, -15681, -15667, -15661, -15659,

-15652, -15640, -15631, -15625, -15454, -15448, -15436, -15435, -15419, -15416, -15408, -15394,

-15385, -15377, -15375, -15369, -15363, -15362, -15183, -15180, -15165, -15158, -15153, -15150,

-15149, -15144, -15143, -15141, -15140, -15139, -15128, -15121, -15119, -15117, -15110, -15109,

-14941, -14937, -14933, -14930, -14929, -14928, -14926, -14922, -14921, -14914, -14908, -14902,

-14894, -14889, -14882, -14873, -14871, -14857, -14678, -14674, -14670, -14668, -14663, -14654,

-14645, -14630, -14594, -14429, -14407, -14399, -14384, -14379, -14368, -14355, -14353, -14345,

-14170, -14159, -14151, -14149, -14145, -14140, -14137, -14135, -14125, -14123, -14122, -14112,

-14109, -14099, -14097, -14094, -14092, -14090, -14087, -14083, -13917, -13914, -13910, -13907,

-13906, -13905, -13896, -13894, -13878, -13870, -13859, -13847, -13831, -13658, -13611, -13601,

-13406, -13404, -13400, -13398, -13395, -13391, -13387, -13383, -13367, -13359, -13356, -13343,

-13340, -13329, -13326, -13318, -13147, -13138, -13120, -13107, -13096, -13095, -13091, -13076,

-13068, -13063, -13060, -12888, -12875, -12871, -12860, -12858, -12852, -12849, -12838, -12831,

-12829, -12812, -12802, -12607, -12597, -12594, -12585, -12556, -12359, -12346, -12320, -12300,

-12120, -12099, -12089, -12074, -12067, -12058, -12039, -11867, -11861, -11847, -11831, -11798,

-11781, -11604, -11589, -11536, -11358, -11340, -11339, -11324, -11303, -11097, -11077, -11067,

-11055, -11052, -11045, -11041, -11038, -11024, -11020, -11019, -11018, -11014, -10838, -10832,

-10815, -10800, -10790, -10780, -10764, -10587, -10544, -10533, -10519, -10331, -10329, -10328,

-10322, -10315, -10309, -10307, -10296, -10281, -10274, -10270, -10262, -10260, -10256, -10254

};

private static string[] pyName = new string[]

{

"A", "Ai", "An", "Ang", "Ao", "Ba", "Bai", "Ban", "Bang", "Bao", "Bei", "Ben",

"Beng", "Bi", "Bian", "Biao", "Bie", "Bin", "Bing", "Bo", "Bu", "Ba", "Cai", "Can",

"Cang", "Cao", "Ce", "Ceng", "Cha", "Chai", "Chan", "Chang", "Chao", "Che", "Chen", "Cheng",

"Chi", "Chong", "Chou", "Chu", "Chuai", "Chuan", "Chuang", "Chui", "Chun", "Chuo", "Ci", "Cong",

"Cou", "Cu", "Cuan", "Cui", "Cun", "Cuo", "Da", "Dai", "Dan", "Dang", "Dao", "De",
 "Deng", "Di", "Dian", "Diao", "Die", "Ding", "Diu", "Dong", "Dou", "Du", "Duan", "Dui",
 "Dun", "Duo", "E", "En", "Er", "Fa", "Fan", "Fang", "Fei", "Fen", "Feng", "Fo",
 "Fou", "Fu", "Ga", "Gai", "Gan", "Gang", "Gao", "Ge", "Gei", "Gen", "Geng", "Gong",
 "Gou", "Gu", "Gua", "Guai", "Guan", "Guang", "Gui", "Gun", "Guo", "Ha", "Hai", "Han",
 "Hang", "Hao", "He", "Hei", "Hen", "Heng", "Hong", "Hou", "Hu", "Hua", "Huai", "Huan",
 "Huang", "Hui", "Hun", "Huo", "Ji", "Jia", "Jian", "Jiang", "Jiao", "Jie", "Jin", "Jing",
 "Jiong", "Jiu", "Ju", "Juan", "Jue", "Jun", "Ka", "Kai", "Kan", "Kang", "Kao", "Ke",
 "Ken", "Keng", "Kong", "Kou", "Ku", "Kua", "Kuai", "Kuan", "Kuang", "Kui", "Kun", "Kuo",
 "La", "Lai", "Lan", "Lang", "Lao", "Le", "Lei", "Leng", "Li", "Lia", "Lian", "Liang",
 "Liao", "Lie", "Lin", "Ling", "Liu", "Long", "Lou", "Lu", "Lv", "Luan", "Lue", "Lun",
 "Luo", "Ma", "Mai", "Man", "Mang", "Mao", "Me", "Mei", "Men", "Meng", "Mi", "Mian",
 "Miao", "Mie", "Min", "Ming", "Miu", "Mo", "Mou", "Mu", "Na", "Nai", "Nan", "Nang",
 "Nao", "Ne", "Nei", "Nen", "Neng", "Ni", "Nian", "Niang", "Niao", "Nie", "Nin", "Ning",
 "Niu", "Nong", "Nu", "Nv", "Nuan", "Nue", "Nuo", "O", "Ou", "Pa", "Pai", "Pan",
 "Pang", "Pao", "Pei", "Pen", "Peng", "Pi", "Pian", "Piao", "Pie", "Pin", "Ping", "Po",
 "Pu", "Qi", "Qia", "Qian", "Qiang", "Qiao", "Qie", "Qin", "Qing", "Qiong", "Qiu", "Qu",
 "Quan", "Que", "Qun", "Ran", "Rang", "Rao", "Re", "Ren", "Reng", "Ri", "Rong", "Rou",
 "Ru", "Ruan", "Rui", "Run", "Ruo", "Sa", "Sai", "San", "Sang", "Sao", "Se", "Sen",

"Seng", "Sha", "Shai", "Shan", "Shang", "Shao", "She", "Shen", "Sheng", "Shi", "Shou", "Shu",

"Shua", "Shuai", "Shuan", "Shuang", "Shui", "Shun", "Shuo", "Si", "Song", "Sou", "Su", "Suan",

"Sui", "Sun", "Suo", "Ta", "Tai", "Tan", "Tang", "Tao", "Te", "Teng", "Ti", "Tian",
 "Tiao", "Tie", "Ting", "Tong", "Tou", "Tu", "Tuan", "Tui", "Tun", "Tuo", "Wa", "Wai",
 "Wan", "Wang", "Wei", "Wen", "Weng", "Wo", "Wu", "Xi", "Xia", "Xian", "Xiang", "Xiao",
 "Xie", "Xin", "Xing", "Xiong", "Xiu", "Xu", "Xuan", "Xue", "Xun", "Ya", "Yan", "Yang",
 "Yao", "Ye", "Yi", "Yin", "Ying", "Yo", "Yong", "You", "Yu", "Yuan", "Yue", "Yun",
 "Za", "Zai", "Zan", "Zang", "Zao", "Ze", "Zei", "Zen", "Zeng", "Zha", "Zhai", "Zhan",

"Zhang", "Zhao", "Zhe", "Zhen", "Zheng", "Zhi", "Zhong", "Zhou", "Zhu", "Zhua", "Zhuai", "Zhuan",

"Zhuang", "Zhui", "Zhun", "Zhuo", "Zi", "Zong", "Zou", "Zu", "Zuan", "Zui", "Zun", "Zuo";

using System.Diagnostics;

Stopwatch watch=new Stopwatch ();

watch.Start();

string time = watch.ElapsedMilliseconds.ToString();

protected void Page_Load(object sender, EventArgs e)

{

int m = System.DateTime.Today.Month;

int y = System.DateTime.Today.Year;

int d = System.DateTime.Today.Day;

int weeks = getWeekDay(y, m, d);

switch (weeks)

{

case 1:

this.TextBox1.Text = "";

```
        break;
    case 2:
        this.TextBox1.Text = "";
        break;
    case 3:
        this.TextBox1.Text = "";
        break;
    case 4:
        this.TextBox1.Text = "";
        break;
    case 5:
        this.TextBox1.Text = "";
        break;
    case 6:
        this.TextBox1.Text = "";
        break;
    case 7:
        this.TextBox1.Text = "";
        break;
    }
}

public static int getWeekDay(int y, int m, int d)
{
    if (m == 1) m = 13;
    if (m == 2) m = 14;
    int week = (d + 2 * m + 3 * (m + 1) / 5 + y + y / 4 - y / 100 + y / 400) % 7
+ 1;
    return week;
}

private static void DeleteFolder(string dir)
{
    foreach (string f in Directory.GetFileSystemEntries(dir))
    {
        if (File.Exists(f))
        {
            FileInfo fi = new FileInfo(f);
            if (fi.Attributes.ToString().IndexOf("ReadOnly") != 1)
            {
                fi.Attributes = FileAttributes.Normal;
            }
            File.Delete(f);
        }
        else
            DeleteFolder(f);
    }
    Directory.Delete(dir);
}

using System;
using System.Collections.Generic;
```


[illegible]

```

1024 / 1024);
        var top2Memory = processes.OrderByDescending(process =>
process.Memory).Take(2)
                                .Sum(process => process.Memory) / 1024 /
1024;
        Console.WriteLine("Memory consumed by the two most hungry
processes: {0} MB", top2Memory);
    }
    public static bool Filter(Process process)
    {
        return process.WorkingSet64 >= 200 * 1024 * 1024;
    }
    static Int64 TotalMemory(this IEnumerable<ProcessData> processes)
    {
        Int64 result = 0;
        foreach (var process in processes)
        {
            result += process.Memory;
        }
        return result;
    }
    public static void AniType()
    {
        var contact = new { Name = "Bob", Age = 8 };
        Console.WriteLine(contact.Name);
        del myd = x => x + 20;
        int j = myd(3);
    }
    public delegate int del(int i);
}

using System;
using System.Data;
using System.Configuration;
using System.Collections;
using System.Web;
using System.Web.Security;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Web.UI.WebControls.WebParts;
using System.Web.UI.HtmlControls;
using System.Data.SqlClient;
public partial class PageTest : System.Web.UI.Page
{
    private DataView m_DataView;
    protected void GridView1_PageIndexChanging(object sender,
GridViewPageEventArgs e)
    {
        GridView1.PageIndex = e.NewPageIndex;
        GridView1.DataSource = m_DataView;
    }
}

```

```

        GridView1.DataBind();
    }
    protected void RadioButton1_CheckedChanged(object sender, EventArgs e)
    {
        GridView1.PagerSettings.Mode = PagerButtons.Numeric;
        GridView1.DataSource = m_DataView;
        GridView1.DataBind();
    }
    protected void RadioButton2_CheckedChanged(object sender, EventArgs e)
    {
        GridView1.PagerSettings.Mode = PagerButtons.NextPrevious;
        GridView1.DataSource = m_DataView;
        GridView1.DataBind();
    }
    protected void Button1_Click(object sender, EventArgs e)
    {
        String sqlconn = "Data Source=WIN-20111118PND;Initial
Catalog=SuperMarket;User ID=sa;Password=123456";
        SqlConnection myConnection = new SqlConnection(sqlconn);
        myConnection.Open();
        SqlCommand myCommand = new SqlCommand("select * from Product",
myConnection);
        SqlDataAdapter Adapter = new SqlDataAdapter();
        Adapter.SelectCommand = myCommand;
        DataSet myDs = new DataSet();
        Adapter.Fill(myDs);
        m_DataView = myDs.Tables[0].DefaultView;
        myConnection.Close();
        RadioButton1.Checked = true;
        GridView1.DataSource = m_DataView;
        GridView1.DataBind();
    }
    protected void Page_Load(object sender, EventArgs e)
    {
    }
}
<%@ Control Language="C#" AutoEventWireup="true"
CodeBehind="FolderSelection.ascx.cs"
Inherits="KevinZhan.Modules.PageDefine.Views.FolderSelection" %>
<ajaxtoolkit:ModalPopupExtender ID="FolderSelectionModalPopupExtender"
runat="server" TargetControlID="ShowButton"
PopupControlID="FolderSelectionPanel" BackgroundCssClass="popupMask"
DropShadow="True"
CancelControlID="CloseImageButton" Drag="True"
PopupDragHandleControlID="FolderSelectionDiv"
Enabled="True" DynamicServicePath="" />
<asp:Button ID="ShowButton" runat="server" Style="display: none;" />
<asp:Panel runat="server" ID="FolderSelectionPanel" Style="display: none;"
CssClass="popContainer">
    <div class="popTitle" id="FolderSelectionDiv" style="width: 410px;">

```

```

        <asp:ImageButton ID="CloseImageButton" CssClass="ibtnClose"
runat="server" ToolTip="Close"
        ImageUrl="~/App_Themes/Default/Images/Layout/close.png"
meta:resourcekey="CloseImageButton" />
        <asp:Label ID="TitleLabel" runat="server" Text="Folder Selection"
meta:resourcekey="TitleLabel"></asp:Label>
    </div>
    <div class="popContent" style="overflow: auto; width: 400px; height: 250px;">
        <asp:UpdatePanel ID="FolderSelectionUpdatePanel" runat="server"
UpdateMode="Conditional">
            <ContentTemplate>
                <asp:TreeView runat="server" ID="FolderSelectionTreeView"
ShowLines="true" NodeIndent="15"
                CssClass="CustomTreeView">
                    <NodeStyle HorizontalPadding="2px" NodeSpacing="0px"
VerticalPadding="2px" />
                    <SelectedNodeStyle BackColor="#BCCFF" />
                </asp:TreeView>

                <div class="marginTop10"></div>
                <asp:HiddenField ID="OnOkScriptHiddenField" runat="server" />
            </ContentTemplate>
        </asp:UpdatePanel>
    </div>
</asp:Panel>
<script type="text/javascript">
    function initializeFolderDialog() {
        $(' #<%
=this.FolderSelectionTreeView.ClientID %>').find("a[href=' javascript:FolderSele
ction_Link()'']").each(function () {
            this.href = "javascript:";
            this.onclick = TreeView_SelectNodeCustom;
        }).dblclick(function () {
            var script = "setFolder(' " + this.title.replace(/[\]/g, "\\") +
"" );";
            var script = $(' #<% =this.OnOkScriptHiddenField.ClientID %>').val();
            if (script) {
                script += "(this.title)";
                eval(script);
            }
            closeFolderDialog();
            return false;
        });
    }
    public static String HideString(String str, int startShowCount, int endShowCount)
throws Exception
    {
        return HideString(str, startShowCount, endShowCount, "*");
    }
    public static String HideString(String str, int startShowCount, int

```

```

endShowCount, String symbol) throws Exception
{
    if (String.IsNullOrWhiteSpace(str) == false && str.length() >
(startShowCount + endShowCount))
    {
        StringBuilder builder = new StringBuilder();
        builder.append(str.substring(0, startShowCount));
        int count = str.length() - startShowCount - endShowCount;
        for (int i = 0; i < count; i++)
        {
            builder.append(symbol);
            builder.Append("*");
        }
        builder.append(str.substring(str.length() - endShowCount,
endShowCount));
        builder.append(str.substring(str.length()-endShowCount, str.length()))
        return builder.toString();
    }
    return str;
}

public static String getDateTimeString(Date date, String format)
{
    if(String.IsNullOrWhiteSpace(format)) { format="yyyy-MM-dd HH:mm:ss"; }
    SimpleDateFormat sdf = new SimpleDateFormat(format);
    String dtString = sdf.format(date);
    return dtString;
}

function TreeView_SelectNodeCustom() {
    var data = <% =this.FolderSelectionTreeView.ClientID %>_Data;
    if (!data) {
        return;
    }
    var targetNode = event.srcElement || event.target;
    var targetNode = this;
    var node;
    if(targetNode.tagName.toUpperCase() == "IMG"){
        targetNode = targetNode.parentNode;
    }
    if(targetNode.id.toString().substr(targetNode.id.toString().length - 1)
== "i"){
        node = targetNode.parentNode.nextSibling.firstChild;
    }
    else{ // text link
        node = targetNode;
    }
    var nodeId = node.id;
    if ((typeof (data.selectedClass) != "undefined") && (data.selectedClass !=
null)) {
        var id = data.selectedNodeID.value;
        if (id.length > 0) {
            var selectedNode = document.getElementById(id);

```

```

        if ((typeof (selectedNode) != "undefined") && (selectedNode !=
null)) {
            WebForm_RemoveClassName(selectedNode,
data.selectedHyperLinkClass); // data.selectedHyperLinkClass = "";

            selectedNode = WebForm_GetParentByTagName(selectedNode,
"TD");

            WebForm_RemoveClassName(selectedNode, data.selectedClass);
            remove img background-color
            WebForm_RemoveClassName(selectedNode.previousSibling,
data.selectedClass);
        }
    }
    WebForm_AppendToClassName(node, data.selectedHyperLinkClass);
    node = WebForm_GetParentByTagName(node, "TD");
    WebForm_AppendToClassName(node, data.selectedClass)
    // add img background-color
    WebForm_AppendToClassName(node.previousSibling,
data.selectedClass);
}
data.selectedNodeID.value = nodeId;
}
function showFolderDialog() {
    initializeFolderDialog();
    $("#<% =this.ShowButton.ClientID %>").click();
    return false;
}
function closeFolderDialog() {
    $('<% =this.CloseImageButton.ClientID %>').click();
    return false;
}
</script>
public ActionResult Img(string t)
{
    if (!string.IsNullOrEmpty(Request.Params["t"]))
    {
        string urlParams = Request.Params["t"];
        string[] urlParamsArray = urlParams.Split('_');
        if (urlParamsArray.Length < 4)
        {
            return Content("");
        }
        int imgWidth = int.Parse(urlParamsArray[0]);
        int imgHeight = int.Parse(urlParamsArray[1]);
        int imgtype = int.Parse(urlParamsArray[2]);
        int imgFormat = int.Parse(urlParamsArray[3]);
        string imgName = urlParamsArray[4];
        string fileSaveUrlPath = string.Empty;
        string imgAdress = string.Empty;
        if (!string.IsNullOrEmpty(imgName))

```

```

        {
            string fileName = string.Format("{0}{1}{2}{3}_{4}",
                imgWidth.ToString(),                imgHeight.ToString(),
                imgtype.ToString(), imgFormat.ToString(), imgName);
            string imgNameNotFormat = fileName.Substring(0,
                fileName.LastIndexOf("."));
            fileSaveUrlPath = string.Format("~/Imgs/{0}/{1}",
                imgName.Substring(0, 4), Int32.Parse(imgName.Substring(4, 2)));
            fileSaveUrlPath = Server.MapPath(fileSaveUrlPath);
            StringUtils.CreateDirectory(fileSaveUrlPath);
            fileSaveUrlPath = string.Format("{0}\\{1}.jpg",
                fileSaveUrlPath, imgNameNotFormat);
            imgAddress = string.Format("/Imgs/{0}/{1}/{2}.jpg",
                imgName.Substring(0, 4), Int32.Parse(imgName.Substring(4, 2)), imgNameNotFormat);
        }
        if (StringUtils.FileExists(fileSaveUrlPath))
        {
            return Redirect(string.Format("{0}{1}", GetSiteDomainName,
                imgAddress));
        }
        else
        {
            imgName = string.IsNullOrEmpty(imgName) ? "0000000000000000" :
                imgName;
            string filePath = string.Format("~/Images/{0}/{1}/{2}",
                imgName.Substring(0, 4), Int32.Parse(imgName.Substring(4, 2)), imgName);
            filePath = Server.MapPath(filePath);
            if (!StringUtils.FileExists(filePath))
            {
                string defaultName = string.Format("{0}{1}{2}{3}",
                    imgWidth.ToString(),
                    imgHeight.ToString(),
                    imgtype.ToString(),
                    imgFormat.ToString());
                filePath = Server.MapPath("~/Imgs/Default/Default.jpg");
                fileSaveUrlPath =
                    Server.MapPath(string.Format("~/Imgs/Default/{0}.jpg", defaultName));
                imgAddress = string.Format("/Imgs/Default/{0}.jpg",
                    defaultName);
            }
            if (!StringUtils.FileExists(fileSaveUrlPath))
            {
                GDIUtils.GetImage(filePath, fileSaveUrlPath, imgWidth,
                    imgHeight, imgtype, imgFormat);
            }
            return Redirect(string.Format("{0}{1}", GetSiteDomainName,
                imgAddress));
        }
    }
    return null;
}

```

```
    }

    package org.onepiece;
import java.text.SimpleDateFormat;
import java.util.Date;
public class StaticClass
{
    public final static String EmptyString = "";
    public final static String MD5 = "MD5";
    public final static String SHA1 = "SHA1";
    public final static String SHA256 = "SHA256";
    public final static String SHA384 = "SHA384";
    public final static String SHA512 = "SHA512";
    public final static String UTF8 = "UTF-8";
    public final static String Unicode = "Unicode";
    public final static String GB2312 = "GB2312";
    public final static String ASCII = "ASCII";
    public final static String GBK = "GBK";
    public final static String yyyy_MM_dd_HH_mm_ss = "yyyy-MM-dd HH:mm:ss";
    public final static String yyyy_MM_dd = "yyyy-MM-dd";
    public final static String HH_mm_ss = "HH:mm:ss";
    public final static int defaultInt = 0;
    public final static long defaultLong = 0L;
    public final static double defaultDouble = 0.0d;
    public final static String mySqlDriver = "com.mysql.jdbc.Driver";
    public final static String mySqlURL =
    public final static String WindowsNewLine = "\n";
    public final static String NewLine = "\r\n";
    public static boolean StringIsNullOrEmpty(String str)
    {
        if (str==null || str.isEmpty()) { return true; }
        return false;
    }
    public static boolean StringIsNullOrWhiteSpace(String str)
    {
        if (str==null || str.isEmpty()) { return true; }
        if (str.trim().length()==0) { return true; }
        return false;
    }
    public static String GetStringOrDefault(String str)
    {
        return GetStringOrDefault(str, "");
    }
    public static String GetStringOrDefault(String str, String defaultValue)
    {
        return StringIsNullOrWhiteSpace(str) ? defaultValue : str;
    }
    public static boolean StringCompare(String str1, String str2)
    {
        if(StringIsNullOrWhiteSpace(str1) || StringIsNullOrWhiteSpace(str2))
        { return false;}
```



```

        return str1.equalsIgnoreCase(str2);
    }
    public static boolean StringCompareIncludeNull(String str1, String str2)
    {
        return
GetStringOrDefault(str1).equalsIgnoreCase(GetStringOrDefault(str2));
    }

    String str = "13147306612";
    String result = HideString(str, 3, 4);
    printIn(result);
    str = "445321198707051030";
    result = HideString(str, 3, 4);
    printIn(result);

    public static String HideString(String str, int startShowCount, int
endShowCount) throws Exception
    {
        return HideString(str, startShowCount, endShowCount, "*");
    }

    public static String HideString(String str, int startShowCount, int
endShowCount, String symbol) throws Exception
    {
        if (StringIsNullOrWhiteSpace(str) == false && str.length() >
(startShowCount + endShowCount))
        {
            StringBuilder builder = new StringBuilder();
            builder.append(str.substring(0, startShowCount));
            int count = str.length() - startShowCount - endShowCount;
            for (int i = 0; i < count; i++)
            {
                builder.append(symbol);
                builder.Append("*");
            }
            builder.append(str.substring(str.length() - endShowCount,
endShowCount));
            builder.append(str.substring(str.length()-endShowCount, str.length()))
            return builder.toString();
        }
        return str;
    }

    public static String getDateTimeString(Date date, String format)
    {
        if(StringIsNullOrWhiteSpace(format)) { format="yyyy-MM-dd HH:mm:ss"; }
        SimpleDateFormat sdf = new SimpleDateFormat(format);
        String dtString = sdf.format(date);
        return dtString;
    }
}

using System;
using System.Reflection;
using System.Collections.Generic;

```

```

using UnityEngine;
using UnityEditor;
public static class SerializedPropertyValueExtension
{
    public static ValueT Value<ValueT>(this SerializedProperty thisSP, string
    propertyName)
    {
        return Value<ValueT>(thisSP.FindPropertyRelative(propertyName));
    }
    public static ValueT Value<ValueT>(this SerializedProperty thisSP)
    {
        Type valueType = typeof(ValueT);
        if (valueType.IsEnum)
            return (ValueT)Enum.ToObject(valueType, thisSP.enumValueIndex);
        if (typeof(Color).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.colorValue;
        else if (typeof(LayerMask).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.intValue;
        else if (typeof(Vector2).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.vector2Value;
        else if (typeof(Vector3).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.vector3Value;
        else if (typeof(Rect).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.rectValue;
        else if (typeof(AnimationCurve).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.animationCurveValue;
        else if (typeof(Bounds).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.boundsValue;
        else if (typeof(Gradient).IsAssignableFrom(valueType))
            return (ValueT)(object)SafeGradientValue(thisSP);
        else if (typeof(Quaternion).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.quaternionValue;
        if (typeof(UnityEngine.Object).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.objectReferenceValue;
        if (typeof(int).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.intValue;
        else if (typeof(bool).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.boolValue;
        else if (typeof(float).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.floatValue;
        else if (typeof(string).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.stringValue;
        else if (typeof(char).IsAssignableFrom(valueType))
            return (ValueT)(object)thisSP.intValue;
        throw new NotImplementedException("Unimplemented      propertyType
        "+thisSP.propertyType+".");
    }
    public static dynamic Value(this SerializedProperty thisSP)
    {
        switch (thisSP.propertyType) {

```

```

case SerializedPropertyType.Integer:
return thisSP.intValue;
case SerializedPropertyType.Boolean:
return thisSP.boolValue;
case SerializedPropertyType.Float:
return thisSP.floatValue;
case SerializedPropertyType.String:
return thisSP.stringValue;
case SerializedPropertyType.Color:
return thisSP.colorValue;
case SerializedPropertyType.ObjectReference:
return thisSP.objectReferenceValue;
case SerializedPropertyType.LayerMask:
return thisSP.intValue;
case SerializedPropertyType.Enum:
int enumI = thisSP.enumValueIndex;
return new KeyValuePair<int, string>(enumI, thisSP.enumNames[enumI]);
case SerializedPropertyType.Vector2:
return thisSP.vector2Value;
case SerializedPropertyType.Vector3:
return thisSP.vector3Value;
case SerializedPropertyType.Rect:
return thisSP.rectValue;
case SerializedPropertyType.ArraySize:
return thisSP.intValue;
case SerializedPropertyType.Character:
return (char)thisSP.intValue;
case SerializedPropertyType.AnimationCurve:
return thisSP.animationCurveValue;
case SerializedPropertyType.Bounds:
return thisSP.boundsValue;
case SerializedPropertyType.Gradient:
return SafeGradientValue(thisSP);
case SerializedPropertyType.Quaternion:
return thisSP.quaternionValue;
default:
throw new NotImplementedException("Unimplemented      propertyType
"+thisSP.propertyType+".");
}
}

```

Access to SerializedProperty's internal gradientValue property getter, in a manner that'll only soft break (returning null) if the property changes or disappears in future Unity revs.

```

static Gradient SafeGradientValue(SerializedProperty sp)
{
BindingFlags instanceAnyPrivacyBindingFlags = BindingFlags.Public |
BindingFlags.NonPublic | BindingFlags.Instance;
PropertyInfo propertyInfo = typeof(SerializedProperty).GetProperty(
"gradientValue",
instanceAnyPrivacyBindingFlags,

```

```
null,
typeof(Gradient),
new Type[0],
null
);
if (propertyInfo == null)
return null;
Gradient gradientValue = propertyInfo.GetValue(sp, null) as Gradient;
return gradientValue;
}
}
using System;
using System.Collections.Generic;
using System.Web;
using System.IO;
using System.Collections;
public class Uploader
{
    string state = "SUCCESS";
    string URL = null;
    string currentType = null;
    string uploadpath = null;
    string filename = null;
    public Hashtable upFile(HttpContext cxt, string pathbase, string[] filetype,
int size)
    {
        pathbase = pathbase + DateTime.Now.ToString("yyyy-MM-dd") + "/";
        uploadpath = cxt.Server.MapPath(pathbase);
        try
        {
            uploadFile = cxt.Request.Files[0];
            originalName = uploadFile.FileName;
            if (checkType(filetype))
            {
                state = "";
            }
            if (checkSize(size))
            {
                state = "";
            }
            if (state == "SUCCESS")
            {
                filename = reName();
                uploadFile.SaveAs(uploadpath + filename);
                URL = pathbase + filename;
            }
        }
    }
}
```